

WORLDLINE GLOBAL SERVICES

SDET / Test Automation Lead Interview Preparation Guide

Tuesday Interview • 13+ Years Experience • Java • Playwright • Selenium • API • CI/CD

Focus: Senior SDET engineering, automation architecture, Java, API testing, SQL, Jenkins/CI-CD, testing strategy, payment-domain scenarios and leadership.

Preparation principle: Answer at framework and engineering level. For every technical answer, explain the problem, design decision, trade-offs, implementation and measurable outcome.

1. Interview Focus Areas

Priority	Area	Expected Focus
1	Java	Core Java, Collections, OOP, coding
2	Automation Framework	Architecture, POM, reusable components, parallelism
3	Playwright / Selenium	Synchronization, locators, flaky tests, Grid
4	API Testing	REST, REST Assured, POJO, ObjectMapper, validation
5	CI/CD	Jenkins, pipelines, reports, quality gates
6	SQL	Joins, subqueries, grouping, database concepts
7	Testing Strategy	Testing Pyramid, risk, automation decisions
8	Leadership	Mentoring, conflicts, production defects, delivery

2. Java Questions

Core Java questions

- What are the four pillars of OOP?
- Abstract class vs interface.
- Method overloading vs overriding.
- final vs finally vs finalize.
- == vs equals().
- Why is String immutable? What is the String pool?
- String vs StringBuilder vs StringBuffer.
- HashMap internal implementation and collision handling.
- HashMap vs Hashtable vs ConcurrentHashMap.
- ArrayList vs LinkedList; HashSet vs LinkedHashSet vs TreeSet.
- Comparable vs Comparator.
- Checked vs unchecked exceptions; throw vs throws.
- Can static methods be overridden? Can they be overloaded?
- Can an abstract class have a constructor?
- Can an interface contain implemented methods?
- What happens when two objects have the same hashCode?

Coding problems to practise

- Reverse a String; reverse words; palindrome; anagram.
- Find duplicate characters/numbers and first non-repeated character.
- Count character/word occurrences.
- Second-largest number; missing number; common array elements.
- Remove duplicates; check whether an array is sorted.
- Find max/min; sort without built-in sort.

- Fibonacci; prime number; factorial; swap two numbers.

Senior-level expectation

Do not stop at working code. Explain time complexity, space complexity, null/empty input, duplicate data, integer overflow where relevant, and how you would make the solution production-ready.

3. Automation Framework Architecture

```

Test Layer
↓
BDD / TestNG / JUnit
↓
Page Objects / API Clients
↓
Reusable Utilities
↓
Browser / Driver Layer
↓
Playwright / Selenium
↓
Application

```

Be prepared to explain

- POM and separation of concerns.
- Reusable browser/driver wrapper.
- Locator management and stable selectors.
- Test data and configuration management.
- API clients and reusable request/response specifications.
- Assertions, logging, screenshots, tracing and reporting.
- Retry strategy without hiding real product defects.
- Parallel execution and environment management.
- CI/CD integration, quality gates and release regression.
- How you migrated Selenium to Playwright and what improved.

Framework-from-scratch question

```

Requirement
↓
Test Strategy
↓
Framework Architecture
↓
Reusable Components
↓
Test Data
↓
Reporting
↓
CI/CD
↓
Quality Gates

```

4. Selenium & Playwright

Selenium questions

- Selenium architecture and WebDriver vs RemoteWebDriver.
- Implicit vs explicit vs fluent wait.
- Why avoid Thread.sleep?
- StaleElementReferenceException and how to solve it.
- ElementClickInterceptedException and NoSuchElementException.
- Dynamic elements, Shadow DOM, frames, alerts and multiple windows.
- Selenium Grid 4 and parallel execution.
- Headless execution, screenshots and retry handling.
- How do you reduce flaky tests?

StaleElementReferenceException — model answer

A stale element occurs when the DOM element represented by a WebElement reference is no longer attached to the current DOM. This commonly happens after a refresh, re-render or AJAX update. I prefer locating the element again using a stable locator rather than keeping an old WebElement reference. In Page Object design, I prefer locator-based access and fresh lookup for dynamic pages.

Why Selenium → Playwright?

- Auto-waiting and built-in assertions reduce synchronization problems.
- Browser/context isolation makes parallel and independent tests easier.
- Tracing, screenshots and videos improve debugging.
- Network interception and API capabilities support broader testing.
- Parallel execution and browser control can improve feedback time.
- The migration should be explained with your actual project outcomes, not only generic tool advantages.

5. API Testing / REST Assured

REST fundamentals

- GET vs POST vs PUT vs PATCH; PUT vs PATCH.
- Idempotency and why it matters for payment/retry scenarios.
- HTTP status codes, headers, path parameters and query parameters.
- Authentication vs authorization; OAuth2 and JWT.
- Request/response validation, JSON schema and contract testing.
- API chaining, negative testing, pagination, rate limiting and retries.
- API performance and resilience testing.

REST Assured example

```
given()
.header("Content-Type", "application/json")
.body(request)
.when()
.post("/users")
.then()
```

```
.statusCode(201);
```

- Be ready to explain POJO serialization/deserialization.
- ObjectMapper and nested POJOs.
- JSONPath and schema validation.
- Request/response specifications and reusable API clients.
- Authentication and environment configuration.

6. SQL / Database

- INNER, LEFT, RIGHT and FULL JOIN.
- WHERE vs HAVING; GROUP BY; DISTINCT.
- UNION vs UNION ALL.
- Subquery vs correlated subquery.
- Primary key, foreign key and indexes.
- Normalisation and transactions.
- ACID properties.

```
SELECT department, COUNT(*)  
FROM employee  
GROUP BY department;
```

```
SELECT MAX(salary)  
FROM employee;
```

```
SELECT *  
FROM employee  
WHERE salary > 50000;
```

7. Jenkins / CI-CD

```
Git Push
↓
Webhook
↓
Jenkins
↓
Checkout
↓
Build
↓
Install dependencies
↓
Run Tests
↓
Generate Report
↓
Publish Results
↓
Quality Gate
```

- Pipeline vs freestyle; declarative vs scripted pipeline.
- Automatic triggers and webhooks.
- Parallel test execution.
- Environment variables and secret management.
- Allure/test-result publishing.
- Smoke vs regression execution.
- Handling multiple environments.
- Git integration and failed-build diagnosis.
- Retry strategy and quality gates.

8. Testing Pyramid

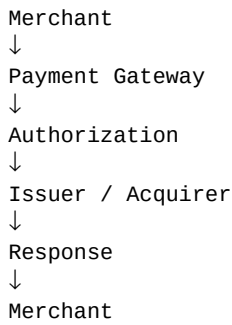
```
/\
/ \
/ E2E \
/-----\
/ API \
/-----\
/ Integration\
/-----\
/ Unit Tests \
/_____ \
```

- Unit tests validate business logic quickly.
- Component/integration tests validate service interactions.
- API tests validate service/business behaviour without UI overhead.
- UI/E2E tests should focus on critical user journeys.
- Do not automate everything through the UI: choose the layer based on risk, stability, speed and ROI.

9. Payment-Domain Scenarios

A payment transaction can be viewed as:

```
Customer
↓
```



How would you test a payment transaction?

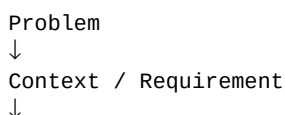
- Happy path and successful authorization.
- Invalid/expired card, wrong CVV, insufficient funds and OTP failure.
- Timeout, network failure and partial failure.
- Duplicate request, double-click and retry/idempotency behaviour.
- Concurrency and transaction ordering.
- Decline, reversal, refund and settlement scenarios.
- Security, audit logging and sensitive-data handling.
- Performance and high-volume transaction scenarios.

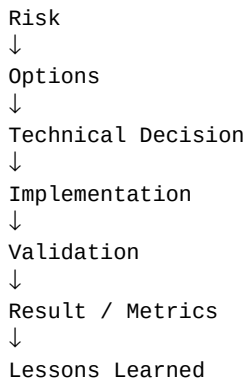
Senior-level point: test failure modes and recovery behaviour, not only the happy path.

10. Senior SDET / Leadership Questions

- How do you decide what should be automated?
- What factors determine automation ROI?
- How do you handle flaky tests?
- A test passes locally but fails in Jenkins. How do you debug it?
- 1,000 tests take 3 hours. How do you reduce execution time?
- The application has no stable IDs. How do you design locators?
- Developers deploy every day. How do you maintain regression coverage?
- How would you design automation for a microservices application?
- How would you test an API when the UI is unavailable?
- How do you handle disagreement with a developer over a defect?
- How do you mentor junior engineers?
- How do you prioritise quality when delivery deadlines are tight?
- Describe a production defect and what you changed afterwards.
- Describe a framework migration and measurable results.
- How do you lead without authority?

11. Scenario Answer Framework





Use this structure for senior-level answers. It keeps answers concise while demonstrating ownership and engineering judgement.

12. 60-Second Introduction

Hi, I'm Swapnil. I'm an SDET and Test Automation Lead with 13+ years of experience in software quality engineering. My core expertise is designing and maintaining scalable automation frameworks for Web, API and mobile applications using Java, JavaScript/TypeScript, Selenium, Playwright, Appium and REST APIs. In my current role, I lead automation for a multi-state healthcare insurance platform. One of my key initiatives was migrating our automation framework from Selenium to Playwright, improving execution stability, reducing flaky tests and making the framework more maintainable. I have strong experience in framework architecture, API automation, CI/CD using Jenkins, Git, Docker and AWS, along with Agile/Scrum practices. Recently, I have also been using AI-assisted development with tools such as Cursor and Claude Code, following repository-level rules and skills to improve automation development rather than relying on ad-hoc prompting. At this stage of my career, I enjoy not only writing automation but also driving the overall quality strategy, mentoring engineers and improving engineering practices around testing.

13. Final Revision Checklist

- Java coding: practise at least 10 problems without IDE assistance.
- Collections: HashMap, Set, List, Comparable/Comparator.
- Framework: be able to draw your architecture from memory.
- Playwright: auto-wait, locators, contexts, tracing, parallelism.
- Selenium: waits, stale element, Grid, dynamic elements.
- API: REST, REST Assured, ObjectMapper, POJO, validation.
- SQL: joins, group by, having, subqueries.
- Jenkins: pipeline flow, parallelism, reports, quality gates.
- Testing Pyramid and automation strategy.
- Payment scenarios: failure modes, retry, idempotency, refund/reversal.
- Prepare 4–5 STAR stories: migration, flaky tests, production issue, conflict, leadership.
- Keep answers tied to your real project experience and quantify outcomes when possible.

Good luck with your Worldline interview!